UNIVERSIDAD ANDRÉS BELLO · UNAB ONLINE

Asignatura APTC101 · Programación orientada a objetos

Sistema de gestión de biblioteca

Sede Antonio Varas

Semana 6 · Sumativa 2: Trabajo grupal – Entrega final

Integrantes

N°	Nombre y apellido
1	Rommy Bejar
2	
3	
4	

Fecha: noviembre de 2026

Índice

1. Introducción 3
2. Análisis y diseño de la solución 4
2.1. Requerimientos y supuestos 4
2.2. Clases, atributos y responsabilidades 5
2.3. Diagrama de clases UML 6
2.4. Fundamentación de las relaciones 7
2.5. Relación con la entrega formativa (semana 4) 7
2.6. Análisis crítico del diagrama de clases 8
3. Aspectos y fundamentos de programación 9
3.1. Tecnología y estructura del proyecto 9
3.2. Colecciones y persistencia en archivos 9
3.3. Validaciones de negocio 9
3.4. Interfaz gráfica y navegación 9
3.5. Flujo de préstamo y devolución 9
3.6. Buenas prácticas de programación 10
3.7. Pruebas de funcionamiento 10
4. Conclusiones 12
5. Bibliografía 13

1. Introducción

La biblioteca de la sede Antonio Varas de la Universidad Andrés Bello necesita administrar sus usuarios, sus libros y los préstamos y devoluciones de forma consistente. Cuando esta gestión es manual o fragmentada aparecen problemas conocidos: usuarios duplicados, errores en el inventario, libros prestados dos veces sin que nadie lo note y multas mal calculadas.

Motivación. El equipo de la biblioteca solicitó actualizar su sistema con requerimientos precisos (usuarios, libros, préstamos, devoluciones y almacenamiento en archivos de texto). El problema es un contexto laboral real, con reglas de negocio claras, y permite aplicar los fundamentos de la programación orientada a objetos (POO): clases, herencia, polimorfismo, encapsulamiento y distintos tipos de relaciones entre clases.

Objetivo del proyecto. Diseñar e implementar en Java una aplicación de escritorio con interfaz Swing y almacenamiento en archivos de texto plano, capaz de registrar estudiantes y docentes, administrar los libros y controlar préstamos y devoluciones según las reglas del enunciado.

Los objetivos específicos son: (1) modelar las entidades con un diagrama de clases UML y sus relaciones; (2) implementar las reglas de validación de RUN, género, ISBN, inventario y plazos; (3) construir una interfaz con JFrame y JDialog que permita volver siempre al inicio; (4) guardar y recuperar los datos en archivos de texto; y (5) comprobar los casos de éxito y de rechazo con pruebas automatizadas.

Objetivo de este informe. Presentar y fundamentar el diseño de la solución (clases, atributos, métodos y relaciones de asociación, agregación, composición, herencia y polimorfismo), explicar las decisiones de programación y mostrar los resultados de las pruebas.

Estructura. El capítulo 2 presenta el análisis y diseño, incluido el diagrama de clases, la fundamentación de las relaciones y un análisis crítico. El capítulo 3 explica los aspectos y fundamentos de programación, con las colecciones, el almacenamiento en archivos, las validaciones, la interfaz y las pruebas. El capítulo 4 contiene las conclusiones y el capítulo 5, la bibliografía.

2. Análisis y diseño de la solución

2.1. Requerimientos y supuestos

Del enunciado se identificaron los siguientes requerimientos funcionales:

- **Usuarios:** alta, edición y eliminación. Cada usuario tiene nombre completo, RUN, género, carrera y préstamo. El RUN no puede repetirse y se valida su formato y dígito verificador; el género debe ser M o F; el préstamo vale 0 si no tiene libro, o el ISBN si lo tiene. Los usuarios son estudiantes o docentes; a los docentes se les registra además la profesión y sus grados.
- **Libros:** ISBN único, título, autor, cantidad en biblioteca, cantidad disponible e imagen. La cantidad en biblioteca debe ser mayor a cero y la disponible debe ser mayor a cero y menor o igual a la anterior. Se pueden crear y eliminar (por ISBN, validando que exista).
- **Préstamo:** el libro y el usuario deben existir, el libro debe tener un ejemplar disponible y el usuario debe estar habilitado. La fecha de inicio es la actual, los días no pueden superar 20 (docente) o 10 (estudiante) y la fecha de devolución se calcula automáticamente. Se descuenta un ejemplar, el usuario queda no habilitado y se genera la tarjeta de préstamo para imprimir.
- **Devolución:** el libro y el usuario deben existir y el ISBN debe coincidir con el préstamo del usuario. Se suma un ejemplar, se habilita al usuario y se cobra una multa de $1.000 por cada día de atraso.
- **Operación:** menú principal con acceso permanente al inicio, interfaz con JFrame y JDialog, y todos los cambios guardados en archivos de texto plano.

Supuesto del grupo: un usuario solo puede tener un préstamo activo a la vez, porque el enunciado representa el préstamo con un único atributo (0 o ISBN).

2.2. Clases, atributos y responsabilidades

Se identificaron diez clases. Los atributos son privados y se accede a ellos mediante métodos públicos, en coherencia con el encapsulamiento.

Clase	Responsabilidad	Atributos y métodos principales
Usuario (abstracta)	Datos comunes de docentes y estudiantes; declara el plazo máximo de forma polimórfica.	- nombreCompleto, run, genero, carrera, isbnPrestado + maximoDiasPrestamo(): int {abstracto} + getTipo(): String {abstracto} + estaHabilitado(): boolean
Estudiante	Especialización de Usuario con plazo de 10 días.	+ maximoDiasPrestamo(): int → 10 + getTipo(): String
Docente	Especialización de Usuario con plazo de 20 días, profesión y grados.	- profesion, grados + maximoDiasPrestamo(): int → 20 + getTipo(): String
Libro	Identidad por ISBN e inventario de ejemplares.	- isbn, titulo, autor, cantidadTotal, cantidadDisponible, imagen + getters y setters, toString()
Prestamo	Registro de un préstamo activo: usuario, libro, plazo y fechas.	- run, isbn, fechaInicio, fechaVencimiento, dias + tarjeta(u, l): String + getters, toString()
BibliotecaServicio	Reglas de negocio: altas, bajas, préstamos, devoluciones y multas.	- usuarios, libros, prestamos (Map), repo + agregarUsuario(), editarUsuario(), eliminarUsuario() + agregarLibro(), eliminarLibro() + prestar(), devolver()
RepositorioTexto	Lectura y escritura de usuarios.txt, libros.txt y prestamos.txt.	- carpeta: Path + leerX() y guardarX() para usuarios, libros y préstamos
Validaciones	Validación de RUN, género, ISBN y campos obligatorios.	+ runNormalizado(), genero(), isbn(), requerido()
MainFrame	Ventana principal (JFrame) con módulos y formularios JDialog.	- servicio, pestañas + formularios, mostrarTarjeta(), refrescar()
Main / Demo	Puntos de entrada: Main abre la interfaz; Demo ejecuta una demostración por consola.	+ main(args): void

2.3. Diagrama de clases UML

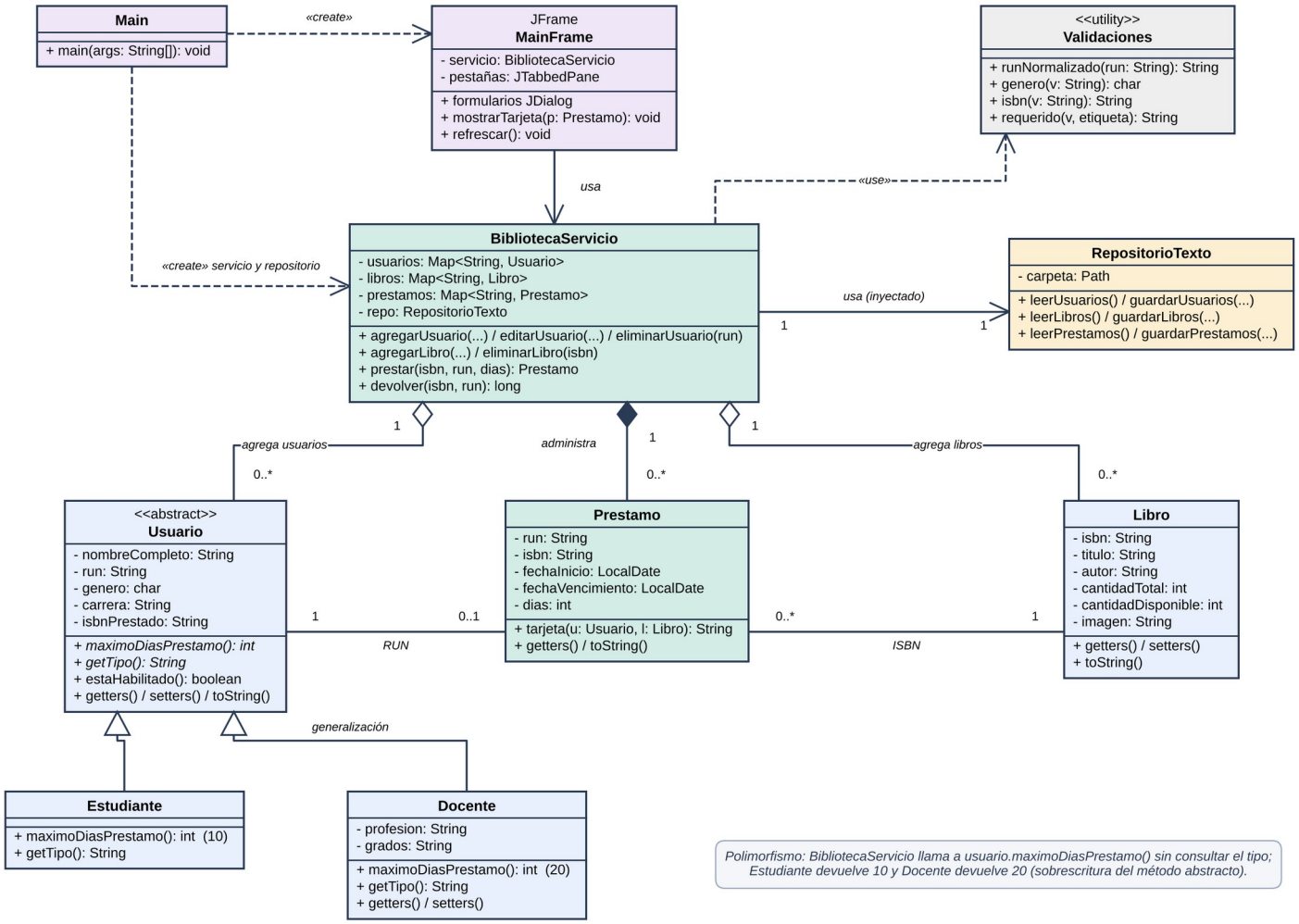


Figura 1. Diagrama de clases final. Rombo lleno: composición; rombo vacío: agregación; triángulo vacío: generalización; línea simple: asociación; flecha punteada: dependencia. Cursiva: elemento abstracto.

2.4. Fundamentación de las relaciones

La tabla resume cada relación del diagrama y la razón de haberla elegido. La notación sigue la especificación UML 2.5.1 (Object Management Group [OMG], 2017).

Tipo	Relación	Mult.	Justificación
Herencia	Estudiante, Docente ▷ Usuario	—	Relación «es-un»: un estudiante y un docente son usuarios. Heredan nombre, RUN, género, carrera y préstamo, y agregan lo propio (el docente, profesión y grados). Usuario es abstracta porque no existe un usuario «genérico» que pueda registrarse.
Polimorfismo	Usuario.maximoDiasPrestamo()	—	El servicio llama a `usuario.maximoDiasPrestamo()` sin preguntar el tipo: Estudiante devuelve 10 y Docente devuelve 20. Agregar un nuevo tipo de usuario no obliga a modificar `prestar()`.
Asociación	Usuario — Prestamo	1 a 0..1	Un préstamo pertenece a un usuario. Se enlazan por el RUN, y cada usuario tiene a lo más un préstamo activo.
Asociación	Prestamo — Libro	0..* a 1	Un préstamo recae sobre un libro, enlazado por el ISBN. Un libro puede tener varios préstamos activos si hay varios ejemplares.
Asociación	BibliotecaServicio → RepositorioTexto	1 a 1	El repositorio se crea en `Main` y se entrega al servicio por el constructor. El servicio lo usa pero no lo posee, por eso es asociación y no composición.
Agregación	BibliotecaServicio ◇— Usuario	1 a 0..*	El servicio reúne usuarios, que son entidades con identidad propia (RUN) y que se cargan desde el archivo; no nacen ni mueren con el servicio.
Agregación	BibliotecaServicio ◇— Libro	1 a 0..*	Igual que con los usuarios: el servicio reúne libros identificados por su ISBN, que existen independientemente de él.
Composición	BibliotecaServicio ◆— Prestamo	1 a 0..*	Un préstamo solo existe dentro del servicio: lo crea `prestar()` y lo elimina `devolver()`. Su ciclo de vida depende del todo, y nadie más lo crea ni lo comparte.
Dependencia	Main ⇢ MainFrame, BibliotecaServicio; BibliotecaServicio ⇢ Validaciones	—	Relaciones de uso o creación sin guardar una referencia permanente: Main crea la ventana y el servicio; el servicio usa los métodos estáticos de Validaciones.

Encapsulamiento. Los atributos de todas las clases son privados. Los cambios de estado pasan por métodos de dominio: por ejemplo, el inventario de un libro y el atributo préstamo del usuario solo cambian dentro de `prestar()` y `devolver()`, que además revierten el cambio si no se logra guardar el archivo.

2.5. Relación con la entrega formativa (semana 4)

En la entrega formativa se modeló una biblioteca genérica con material bibliográfico abstracto (Libro y Revista), ejemplares, catálogo y préstamos, para practicar las cinco relaciones. Para esta entrega final se rediseñó el modelo a partir del enunciado oficial de la sede Antonio Varas, que

exige otros conceptos: usuarios de dos tipos, libros con inventario, plazos, multas y almacenamiento en archivos.

Se conservaron los criterios de diseño de la formativa: una clase abstracta con herencia y polimorfismo (de `MaterialBibliografico.calcularPlazo()` a `Usuario.maximoDiasPrestamo()`), composición solo cuando hay dependencia de ciclo de vida, agregación para colecciones de entidades con identidad propia, y una única fuente de verdad para el estado. En esta versión, el atributo préstamo del usuario, el stock del libro y el registro del préstamo se modifican juntos en el servicio y se revierten si falla el guardado.

2.6. Análisis crítico del diagrama de clases

1. **Usuario abstracta con dos subclases.** Estudiante y Docente comparten casi todos sus datos y solo difieren en el plazo y en la profesión y los grados. La herencia evita duplicar el código y el polimorfismo elimina los condicionales por tipo (Fowler, 2004).
2. **Reglas de negocio en BibliotecaServicio y no en la interfaz.** La ventana solo muestra datos y captura entradas. Así las reglas se pueden probar sin abrir la interfaz y se podrían reutilizar con otra (consola, web).
3. **Un préstamo por usuario, guardado en un Map por RUN.** Es la forma más directa de cumplir la regla de que el usuario queda no habilitado mientras tiene un libro, y permite buscar el préstamo en tiempo constante.
4. **Prestamo guarda el RUN y el ISBN en lugar de referencias a objetos.** Se decidió así porque los datos se guardan en texto plano y se recargan al iniciar: guardar claves evita duplicar los objetos completos y que queden desactualizados. La contrapartida es que para mostrar la tarjeta hay que buscar el usuario y el libro en el servicio. Una alternativa sería guardar referencias y reconstruirlas al leer los archivos.
5. **Composición y agregación.** Los préstamos se componen en el servicio porque no existen fuera de él. Los usuarios y libros se agregan porque tienen identidad propia. El repositorio es una asociación porque se inyecta desde afuera. Varios autores señalan que la agregación aporta poca semántica adicional a una asociación (Fowler, 2004); aquí se usa para expresar de forma explícita la idea de colección.

Limitaciones. (a) `carrera` está en Usuario porque el enunciado la lista para todos los usuarios, aunque para un docente es opcional. (b) Los tres archivos se guardan de forma independiente: cada uno se reemplaza de forma segura, pero no hay una transacción conjunta. (c) El ISBN se valida por estructura (10 o 13 caracteres), no por su dígito de control. (d) La imagen del libro se guarda como ruta y no se muestra en pantalla.

3. Aspectos y fundamentos de programación

3.1. Tecnología y estructura del proyecto

El proyecto está desarrollado en Java SE 11, con la API Swing para la interfaz, `java.time` para las fechas y `java.nio.file` para los archivos. El código fuente está en `BibliotecaUNAB/src/cl/biblioteca` y se organiza en paquetes: `modelo` (Usuario, Estudiante, Docente, Libro, Prestamo), `servicio` (BibliotecaServicio), `persistencia` (RepositorioTexto), `util` (Validaciones), `ui` (MainFrame) y `prueba` (PruebaLogica). Las clases `Main` y `Demo` son los puntos de entrada.

3.2. Colecciones y persistencia en archivos

El repositorio lee cada archivo con listas `ArrayList` y el servicio las indexa en mapas `LinkedHashMap` (`Map<String, Usuario>`, `Map<String, Libro>` y `Map<String, Prestamo>`). Los mapas permiten comprobar de inmediato que el RUN y el ISBN no se repitan y encontrar el préstamo de un usuario, y conservan el orden de ingreso.

Los datos se guardan en `usuarios.txt`, `libros.txt` y `prestamos.txt`, con un registro por línea, campos separados por tabulaciones y los caracteres especiales escapados. Cada archivo se escribe primero en un archivo temporal y luego reemplaza al original, para reducir el riesgo de corrupción si el programa se interrumpe. Los datos se leen al iniciar y se guardan en cada alta, edición, baja, préstamo o devolución.

3.3. Validaciones de negocio

- **RUN:** se quitan puntos y guion, se valida el formato y el dígito verificador con el módulo 11 (incluidos K y 0). No se permiten RUN repetidos.
- **Usuario:** el género solo puede ser M o F. El estudiante debe tener carrera y el docente, profesión. El atributo préstamo vale "0" sin libro o el ISBN con libro.
- **Libro:** el ISBN debe tener 10 caracteres (el último puede ser X) o 13 dígitos, y ser único. La cantidad en biblioteca debe ser mayor a cero y la disponible, mayor a cero y no superior a la anterior.
- **Préstamo:** el libro y el usuario deben existir, debe haber ejemplares, el usuario debe estar habilitado y los días deben estar entre 1 y el máximo de su tipo. Se descuenta un ejemplar y se guarda.
- **Devolución:** el ISBN debe coincidir con el préstamo activo del usuario. Se repone el ejemplar, se habilita al usuario y la multa es la cantidad de días de atraso por $1.000.

3.4. Interfaz gráfica y navegación

`MainFrame` hereda de JFrame y muestra una pantalla de inicio con acceso a Usuarios, Libros y Préstamos y devoluciones. El botón «Volver al inicio» está disponible en todo momento. Los formularios de usuarios y libros son JDialog modales, y el préstamo y la devolución usan cuadros de diálogo con confirmación. Los errores de validación se informan con mensajes. La tarjeta del préstamo se muestra en un área de texto con la opción de imprimirla.

3.5. Flujo de préstamo y devolución

1. Se ingresan el ISBN y el RUN, que se normalizan y validan.

2. Se comprueba que el libro exista y tenga un ejemplar, y que el usuario exista y esté habilitado.
3. Se verifica que los días no superen `usuario.maximoDiasPrestamo()` (10 o 20).
4. Se registra el inicio con `LocalDate.now()` y se calcula el vencimiento con `plusDays()`.
5. Se asigna el ISBN al usuario, se descuenta un ejemplar y se guardan los tres archivos. Si el guardado falla, los cambios se revierten.
6. Se genera la tarjeta con los datos del préstamo para imprimirla. En la devolución se repone el ejemplar, se habilita al usuario y se informa la multa.

3.6. Buenas prácticas de programación

- **Nombres y convenciones:** clases en `PascalCase`, métodos y atributos en `camelCase` y constantes en mayúsculas. Los nombres describen su función (Martin, 2008).
- **Clases completas:** cada clase del modelo tiene atributos privados, constructor, getters, setters donde corresponde y `toString()`.
- **Modularidad:** métodos pequeños con una sola tarea y un retorno específico (`boolean` para consultas, `long` para la multa, un objeto cuando se crea algo). Las validaciones están centralizadas en `Validaciones`.
- **Documentación:** las clases y los métodos principales tienen comentarios que explican su propósito, y el código está indentado y ordenado por paquetes.
- **Robustez:** los errores de las reglas se informan con excepciones y mensajes claros, y los cambios se revierten si falla el guardado.

3.7. Pruebas de funcionamiento

Se escribió la clase `PruebaLogica`, que ejecuta 55 comprobaciones automáticas sobre la lógica de negocio usando carpetas temporales, sin afectar los datos reales. Todas resultaron correctas. La clase `Demo` ejecuta además un recorrido por consola que crea los objetos y muestra cada caso de éxito y de rechazo. Los resultados completos están en `resultados_pruebas.txt` y `salida_demo.txt`.

Prueba	Resultado obtenido
RUN duplicado o con dígito verificador incorrecto	Rechazado
Género distinto de M o F	Rechazado
ISBN de 10 o 13 caracteres válidos	Aceptado
ISBN de 9, 11 o 12 caracteres, o 13 con letras	Rechazado
Cantidad disponible mayor que la total, o igual a cero	Rechazado
Estudiante pide 11 días / docente pide 21 días	Rechazado (máximos 10 y 20)
Estudiante pide 10 días / docente pide 20 días	Aceptado (polimorfismo)
Préstamo válido	Registra el vencimiento, descuenta el ejemplar y deja al usuario no habilitado
Segundo préstamo del mismo usuario	Rechazado
Libro sin ejemplares / usuario o libro inexistente	Rechazado
Devolución con ISBN distinto al prestado	Rechazado

Prueba	Resultado obtenido
Devolución con 3 días de atraso	Multa de $3.000
Devolución a tiempo	Sin multa; repone el ejemplar y habilita al usuario
Eliminar usuario con préstamo o libro prestado	Rechazado
Reinicio del servicio y eliminaciones	Los datos y las eliminaciones persisten en los archivos

Las pruebas automáticas cubren la capa de negocio. La interfaz gráfica se comprueba ejecutando el proyecto desde NetBeans y recorriendo los módulos de usuarios, libros y préstamos.

4. Conclusiones

Resultados. Se diseñó e implementó una aplicación Java para la biblioteca de la sede Antonio Varas que cubre los requerimientos de usuarios, libros, préstamos y devoluciones, con interfaz Swing y almacenamiento en archivos de texto. El diseño aplica herencia (Usuario, Estudiante y Docente), polimorfismo (el plazo máximo de préstamo), asociación, agregación, composición y encapsulamiento, y reparte las responsabilidades entre el modelo, el servicio, la persistencia, las validaciones y la interfaz.

Nivel de logro de los objetivos. Se cumplieron el modelado con UML, la implementación de las reglas de validación y la persistencia en archivos. Las pruebas automáticas (55 comprobaciones) confirman los casos de éxito y de rechazo de la lógica de negocio. El objetivo de la interfaz se cumple con un menú de inicio, formularios JDialog y acceso permanente al home.

Capacidad crítica. El diseño tiene límites que se identificaron: los tres archivos no se guardan en una transacción conjunta, el ISBN no verifica su dígito de control y el préstamo guarda claves en lugar de referencias. Estas decisiones simplifican la persistencia en texto plano, pero se pueden mejorar.

Marco de acción futura. Como trabajo futuro se propone migrar el almacenamiento a una base de datos con transacciones, validar el dígito de control del ISBN, guardar el historial de préstamos devueltos y las multas acumuladas, mostrar la imagen del libro, permitir varios préstamos por usuario y agregar pruebas automáticas de la interfaz.

5. Bibliografía

Fowler, M. (2004). *UML distilled: A brief guide to the standard object modeling language* (3.ª ed.). Addison-Wesley.

Martin, R. C. (2008). *Clean code: A handbook of agile software craftsmanship*. Prentice Hall.

Object Management Group. (2017). *OMG Unified Modeling Language (OMG UML), Version 2.5.1*. https://www.omg.org/spec/UML/2.5.1

Oracle. (s. f.). *Java Platform, Standard Edition 11: API specification (Swing, java.time, java.nio.file)*. https://docs.oracle.com/en/java/javase/11/docs/api/

Universidad Andrés Bello. (2026). *APTC101, Semana 6: Sumativa 2, trabajo grupal entrega final* [Instrucciones, enunciado y rúbrica de la actividad]. UNAB Online.